

PLAN DE IMPLEMENTACION

Entrega 0.1 - Architectural Baseline

Documento: Plan detallado de implementacion

Version: 1.0

Fecha: 5 de septiembre de 2026

Branch: feat/architectural-baseline

Referencia: hoja_ruta_universal_business_core.pdf

Universal Business Core - Monolito modular, dominio agnostico, multi-tenant

Espera aprobacion antes de proceder a la implementacion.

Documento: Plan detallado de implementación (revisado v2.0 - correcciones arquitectónicas) Versión: 2.0 (1ª revisión de plan, 18 correcciones arquitectónicas aplicadas) Fecha: 5 de septiembre de 2026 Branch: [feat/architectural-baseline] Documento de autoridad: [hoja_ruta_universal_business_core.pdf] Estado: Espera de aprobación arquitectónica antes de implementar código

A. Evaluación actual del repositorio

Estado actual: Repositorio esencialmente vacío.

Elemento	Estado
Código Python / paquete [src/universal_business/]	? Inexistente
[pyproject.toml]	? Inexistente
Estructura de paquetes y módulos	? Inexistente
Tests ([tests/])	? Inexistentes
Documentación arquitectónica ([ARCHITECTURE.md], ADRs)	? Inexistente
Configuración CI ([.github/workflows/ci.yml])	? Inexistente
[.gitignore]	? Existente (cubre venv, cachés, logs, .env, sqlite)
Branch actual	? [feat/architectural-baseline] (protegido master)

Activos disponibles:

- [.gitignore] - listo para usar.
- [hoja_ruta_universal_business_core.pdf] - documento de autoridad (12 pp.).
- [docs/plan_entrega_0.1_architectural_baseline.md] - este documento (fuente de verdad de la entrega).

B. Árbol de directorios PROPUESTO (versión CORREGIDA y mínima)

> Principio anti-inflación (Corrección 6): No se crean archivos vacíos. Cada archivo listado cumple AL MENOS UNA de estas funciones: contiene entidad/VO útil, define un contrato real, marca límite arquitectónico, contiene test útil o es `__init__.py` estrictamente necesario para marcar paquete.

```

app_pedidos_pyme/
??? .gitignore # [ya existe]
??? hoja_ruta_universal_business_core.pdf # [ya existe]
??? pyproject.toml # <- metadata + deps + pytest/ruff/mypy
?
??? docs/
?   ??? ARCHITECTURE.md # <- 4 capas + tenancy + extensiones verticales
?   ??? plan_entrega_0.1_architectural_baseline.md # <- ESTE DOCUMENTO
?   ??? plan_entrega_0.1_architectural_baseline.pdf # <- versión PDF (auto-generado)
?   ??? adr/
?       ??? ADR-001-modular-monolith-vs-microservices.md
?       ??? ADR-002-multi-tenancy-strategy.md
?       ??? ADR-003-catalog-item-model.md
?       ??? ADR-004-domain-events-and-outbox.md
?       ??? ADR-005-vertical-extension-model.md
?       ??? ADR-006-resources-and-availability-model.md # <- NUEVO (Corrección 1)
?       ??? ADR-007-idempotency-for-orders-and-reservations.md # <- NUEVO (Corrección 1)
?
??? src/
?   ??? universal_business/
?       ??? __init__.py # marcador de paquete
?       ?
?       ??? domain/ # ? PURO - sin imports de infra/api/verticals
?       ?   ??? __init__.py
?       ?   ?
?       ?   ??? shared/ # Utilidades TRANSVERSALES de dominio
?       ?   ?   ??? __init__.py
?       ?   ?   ??? value_objects/
?       ?   ?   ?   ??? __init__.py
?       ?   ?   ?   ??? ids.py # IDs fuertes (TenantId, BusinessId, LocationId,
CustomerId, EntityId[T])
?       ?   ?   ?   ??? money.py # Money (Decimal + Currency) + política
redondeo/precisión
?       ?   ?   ?   ??? temporal.py # DateRange / TimeRange / datetime timezone-aware
guards
?       ?   ?   ?   ??? status.py # StatusTransition (utilidad), NO enum universal
?       ?   ?   ??? entities.py # BaseEntity + AggregateRoot (SI aportan valor
real: id, timestamps, events)
?       ?   ?   ??? errors.py # DomainError + StatusTransitionError + MoneyError
+ TemporalError + TenantBoundaryError
?       ?   ?   ??? events.py # DomainEvent base (mínimo útil)
?       ?   ?
?       ?   ??? business/ # Tenant / Business / Location
?       ?   ?   ??? __init__.py
?       ?   ?   ??? entities.py # Tenant (límite SaaS), Business, Location
?       ?   ?   ??? value_objects.py # Address, ContactInfo, OperatingHours,
BusinessSettings
?       ?   ?   ??? ports.py # ITenantRepository - IBusinessRepository -
ILocationRepository (Protocol)
?       ?   ?
?       ?   ??? customers/ # Customer + CustomerStatus
?       ?   ?   ??? __init__.py
?       ?   ?   ??? entities.py # Customer, ContactPoint, Consent

```

```

?      ?      ?      ??? value_objects.py      # CustomerPreferences, ConsentType, CustomerStatus
enum
?      ?      ?      ??? ports.py      # ICustomerRepository (Protocol)
?      ?      ?
?      ?      ??? catalog/      # CONTRATOS SOLAMENTE (sin entidades placeholder)
?      ?      ?      ??? __init__.py      # marcador de paquete (establece límite de módulo)
?      ?      ?      ??? ports.py      # ICatalogRepository (contrato, por ahora mínimo)
?      ?      ?
?      ?      ??? resources/      # CONTRATOS SOLAMENTE
?      ?      ?      ??? __init__.py
?      ?      ?      ??? ports.py      # IResourceRepository
?      ?      ?
?      ?      ??? availability/      # CONTRATOS SOLAMENTE
?      ?      ?      ??? __init__.py
?      ?      ?      ??? ports.py      # IAvailabilityRepository
?      ?      ?
?      ?      ??? reservations/      # CONTRATOS + ReservationStatus (solo enum)
?      ?      ?      ??? __init__.py
?      ?      ?      ??? value_objects.py      # ReservationStatus (enum propio del módulo)
?      ?      ?      ??? ports.py      # IReservationRepository
?      ?      ?
?      ?      ??? orders/      # CONTRATOS + OrderStatus (solo enum)
?      ?      ?      ??? __init__.py
?      ?      ?      ??? value_objects.py      # OrderStatus (enum propio del módulo)
?      ?      ?      ??? ports.py      # IOrderRepository
?      ?      ?
?      ?      ??? fulfillment/      # CONTRATOS + FulfillmentStatus (solo enum)
?      ?      ??? __init__.py
?      ?      ??? value_objects.py      # FulfillmentStatus (enum propio del módulo)
?      ?      ??? ports.py      # IFulfillmentRepository
?      ?
?      ??? application/      # Casos de uso (FASE 1 en adelante) - en 0.1: SOLO
marcadores de paquete
?      ?      ??? __init__.py      # marcador (límite arquitectónico: domain <--
application)
?      ?      # NOTA: NO existe
application/ports/repositories.py (Corrección 7)
?      ?
?      ??? infrastructure/      # SIN IMPLEMENTAR - solo marcador
?      ?      ??? __init__.py      # límite: solo marcador, nada de
SQLAlchemy/Postgres
?      ?
?      ??? api/      # SIN IMPLEMENTAR - solo marcador
?      ?      ??? __init__.py      # límite: NO FastAPI, no endpoints
?      ?
?      ??? verticals/      # SIN IMPLEMENTAR - solo marcador
?      ??? __init__.py      # límite: NO pica-pollo, NO nombres sectoriales
?
??? tests/
    ??? __init__.py
    ??? conftest.py      # Fixtures base: TENANT_ID / BUSINESS_ID /
LOCATION_ID / CUSTOMER_ID / TZ / CURRENCY
?
    ??? architecture/      # B. Tests arquitectónicos (Corrección 14.B)

```

```

?   ??? __init__.py
?   ??? test_architecture_boundaries.py      # Reglas: dom->infra? dom->api? dom->FastAPI?
dom->SQLAlchemy? pica-pollo names?
?
??? imports/                                # C. Tests de importación (Corrección 14.C)
?   ??? __init__.py
?   ??? test_import_without externals.py     # universal_business se importa sin servicios
externos
?
??? unit/                                    # A. Tests unitarios (Corrección 14.A)
??? __init__.py
?
??? domain/
    ??? __init__.py
    ?
    ??? shared/
        ?   ??? __init__.py
        ?   ??? test_ids.py
        ?   ??? test_money.py
        ?   ??? test_temporal.py
        ?   ??? test_status_transition.py    # NO LifecycleStatus; StatusTransition utility +
cada dominio su propio enum
        ?   ??? test_domain_events.py
        ?   ??? test_base_entities.py       # BaseEntity / AggregateRoot (valor real:
timestamps, events)
        ?
        ??? business/
            ?   ??? __init__.py
            ?   ??? test_tenant.py          # Tenant NO es necesariamente entidad legal
            ?   ??? test_business.py
            ?   ??? test_location.py
            ?
            ??? customers/
                ??? __init__.py
                ??? test_customer.py        # location_id NO obligatorio

??? .github/
    ??? workflows/
        ??? ci.yml                          # MÍNIMO: ruff check, mypy, pytest

```

Notas importantes sobre el árbol (Corrección 6 aplicada)

- NO existen [entities.py] vacíos en [catalog/resources/availability]. Si un módulo no tiene entidades reales para la 0.1, solo trae [__init__.py] (marcador de límite arquitectónico) y [ports.py] (contrato real para FASE 1).
- [reservations/orders/fulfillment] sí tienen [value_objects.py] porque necesitan exponer sus propios [*Status] enums (Corrección 2).
- [application/] NO contiene subcarpeta [ports/] (Corrección 7). Los contratos viven en [domain/<modulo>/ports.py] y cada caso de uso importa solo los que necesita.

C. Archivos a crear (resumen CORREGIDO por categoría)

> Inflación cero: cada archivo tiene propósito explícito. El número baja de ~63 a ~49.

Configuración raíz (2 archivos)

1. [pyproject.toml] - metadata + deps + pytest/ruff/mypy (sin bandit, sin import-linter, sin pre-commit hard requirement)
2. [.github/workflows/ci.yml] - pipeline MÍNIMO (ruff + mypy + pytest)
3. [.pre-commit-config.yaml] - opcional (se crea solo si aporta valor; si no, se omite)

Documentación (9 archivos)

1. [docs/ARCHITECTURE.md]
2. [docs/adr/ADR-001-modular-monolith-vs-microservices.md]
3. [docs/adr/ADR-002-multi-tenancy-strategy.md]
4. [docs/adr/ADR-003-catalog-item-model.md]
5. [docs/adr/ADR-004-domain-events-and-outbox.md]
6. [docs/adr/ADR-005-vertical-extension-model.md]
7. [docs/adr/ADR-006-resources-and-availability-model.md] - NUEVO (Corrección 1)
8. [docs/adr/ADR-007-idempotency-for-orders-and-reservations.md] - NUEVO (Corrección 1)

Paquete fuente `src/universal\_business/` (~27 archivos, reducido)

Shared / Domain base:

1. [src/universal_business/__init__.py]
2. [domain/__init__.py]
3. [domain/shared/__init__.py]
4. [domain/shared/value_objects/__init__.py]
5. [domain/shared/value_objects/ids.py] - IDs fuertes (wrapper dataclass, runtime safe)
6. [domain/shared/value_objects/money.py] - Money Decimal + Currency (Corrección 5)
7. [domain/shared/value_objects/temporal.py] - DateRange/TimeRange + tz guards
8. [domain/shared/value_objects/status.py] - Solo StatusTransition (NO enum universal, Corrección 2)
9. [domain/shared/entities.py] - BaseEntity + AggregateRoot (valor real: timestamps + events, Corrección 12)
10. [domain/shared/errors.py] - DomainError + transición + moneda + temporal + boundary
11. [domain/shared/events.py] - DomainEvent base mínimo útil

business (completo, 3):

1. [domain/business/__init__.py]
2. [domain/business/entities.py] - Tenant / Business / Location
3. [domain/business/value_objects.py] - Address, ContactInfo, OperatingHours, BusinessSettings
4. [domain/business/ports.py] - 3 Protocolos

customers (completo, 3):

1. [domain/customers/__init__.py]
2. [domain/customers/entities.py] - Customer / ContactPoint / Consent
3. [domain/customers/value_objects.py] - CustomerPreferences, ConsentType, CustomerStatus (propio enum)
4. [domain/customers/ports.py] - ICustomerRepository Protocol

6 módulos con CONTRATOS MÍNIMOS (2 archivos cada uno = 12): 31-42. [catalog/], [resources/], [availability/] -> [__init__.py] + [ports.py] (solo contrato) 43-48. [reservations/], [orders/], [fulfillment/] -> [__init__.py] + [value_objects.py] (Status enum) + [ports.py]

Marcadores de capa (3):

- 1. [application/__init__.py]
- 2. [infrastructure/__init__.py]
- 3. [api/__init__.py]
- 4. [verticals/__init__.py]

Tests (~15 archivos)

- 1. [tests/__init__.py]
- 2. [tests/conftest.py]

55-56. [tests/architecture/] (test + init) 57-58. [tests/imports/] (test + init) 59-70. [tests/unit/domain/shared/] - 6 archivos: ids, money, temporal, status_transition, domain_events, base_entities 71-74. [tests/unit/domain/business/] - 3 + init 75-76. [tests/unit/domain/customers/] - test + init

ESTIMACIÓN FINAL: ~51 archivos (solo aquellos que aportan valor real).

D. Responsabilidades de cada módulo

D.1 `domain/shared/` - Núcleo transversal (VALOR REAL)

Submódulo	Responsabilidad
[value_objects/ids.py]	Wrapper dataclass immutable por tipo ([TenantId], [BusinessId], [LocationId], [CustomerId], [EntityId[T]]). Validación runtime (UUID v4 o UUID str válido). NO [NewType] (sin seguridad runtime).
[value_objects/money.py]	[Money] = [Decimal] + currency code ISO-4217. Admite [int] y [Decimal] en operaciones (NO [float]). Política de redondeo, precisión, comportamiento mezcla monedas. Ver sección F.2 completa.
[value_objects/temporal.py]	[DateRange] (fecha local inclusiva), [TimeRange] (ambos extremos timezone-aware OBLIGATORIO). Guards: [require_aware(dt)] lanza [TemporalRangeError] si [tzinfo is None]. Helpers: [overlap()], [as_utc()], [same_timezone()].

[value_objects/status.py]	SOLAMENTE UTILIDADES. [StatusTransition.validate(from, to, valid_map)] + [TransitionGuard]. NO enum universal de estados. Cada dominio define SU PROPIO [XxxStatus(str, Enum)].
[entities.py]	[BaseEntity] y [AggregateRoot] SI aportan valor real (Corrección 12): identidad tipada, [created_at: datetime] UTC-aware, [updated_at: datetime] UTC-aware, igualdad por ID. [AggregateRoot] extiende BaseEntity y añade colección inmutable de [DomainEvent] con [record_event()/events()/clear_events()]. NO es herencia cosmética.
[errors.py]	Jerarquía [DomainError] -> [InvariantViolation], [StatusTransitionError], [MoneyCurrencyMismatchError], [MoneyRoundingError], [TemporalRangeError], [TenantBoundaryViolationError], [IdempotencyViolationError].
[events.py]	[DomainEvent] base sencillo y útil (Corrección 13): [event_id], [occurred_at] (UTC aware), [aggregate_id], [aggregate_type] (OBLIGATORIOS). Opcionales cuando procedan: [tenant_id], [business_id], [location_id], [metadata].

D.2 `domain/business/` - Jerarquía tenancy (Tenant = límite SaaS)

Archivo	Responsabilidad
[entities.py]	[Tenant] = límite superior de aislamiento SaaS (Corrección 4). Representa cuenta/holding/franquicia/organización; NO necesariamente entidad legal. Contiene [tenant_id] (pk), [name] (nombre visible), [legal_name] (opcional, solo si hay persona jurídica), [status] (enum [TenantStatus] propio). [Business] = unidad operativa subordinada a Tenant. [Location] = establecimiento/lugar subordinado a Business.

[value_objects.py]	[Address] (país ISO-3166 alpha-2 obligatorio), [ContactInfo] (email/teléfono/web), [OperatingHours] (7 días x TimeRange opcionales por día), [BusinessSettings] (default_currency, feature_flags mínimo: [has_reservations], [has_delivery]).
[ports.py]	[ITenantRepository] (get, list), [IBusinessRepository] (get, list_by_tenant), [ILocationRepository] (get, list_by_business). Protocol (NO ABC). Cada método requiere contexto tenant_id cuando le corresponde.

D.3 `domain/customers/` - Cliente (location_id NO obligatorio, Corrección 3)

Archivo	Responsabilidad
[entities.py]	[Customer]: [customer_id], [tenant_id] (OBLIGATORIO), [business_id] (OBLIGATORIO), [location_id] (OPCIONAL, NO parte de la identidad). Un customer puede relacionarse con múltiples Locations del mismo Business/Tenant sin duplicarse. [given_name], [family_name?], [full_name] computed, [contact_points: list[ContactPoint]], [addresses: list[Address]], [consents], [preferences], [status: CustomerStatus].
[value_objects.py]	[CustomerPreferences] (idioma ISO 639-1, canal notificado, retención datos), [ConsentType] enum (marketing, profiling, terms, data_sharing), [CustomerStatus(str, Enum)] = DRAFT / ACTIVE / SUSPENDED / ANONYMIZED / ARCHIVED (propio del módulo).
[ports.py]	[ICustomerRepository] -> [get(customer_id)], [get_by_external_ref(tenant_id, business_id?, external_ref)], [search(tenant_id, business_id?, query)].

D.4 Módulos de dominio verticales (catalog/resources/availability/reservations/orders/fulfillment)

> Corrección 6 aplicada: Mínimo viable para 0.1 - solo contratos y enums de estado necesarios. NO entidades placeholder.

Módulo	Archivos en 0.1	Qué aportan
[catalog/]	[__init__.py] + [ports.py]	Establece límite arquitectónico del módulo; define [ICatalogRepository] Protocol mínimo para FASE 1.
[resources/]	[__init__.py] + [ports.py]	Idem; [IResourceRepository] - en FASE 1 contendrá Resource/ResourceType.
[availability/]	[__init__.py] + [ports.py]	Idem; [IAvailabilityRepository] - reglas de disponibilidad en FASE 1.
[reservations/]	[__init__.py] + [value_objects.py] + [ports.py]	[ReservationStatus(str, Enum)] propio: DRAFT / CONFIRMED / CHECKED_IN / COMPLETED / CANCELLED / NO_SHOW. Contrato [IReservationRepository].
[orders/]	[__init__.py] + [value_objects.py] + [ports.py]	[OrderStatus(str, Enum)] propio: DRAFT / VALIDATED / CONFIRMED / PREPARING / READY / DELIVERING / COMPLETED / CANCELLED / REFUNDED. Contrato [IOrderRepository].
[fulfillment/]	[__init__.py] + [value_objects.py] + [ports.py]	[FulfillmentStatus(str, Enum)] propio: PENDING / ASSIGNED / IN_PROGRESS / COMPLETED / FAILED / CANCELLED. Contrato [IFulfillmentRepository].

D.5 `application/`

Solo [__init__.py].

- Corrección 7 aplicada: NO existe [application/ports/repositories.py] (punto central de acoplamiento). Cada caso de uso (FASE 1) importará directamente [from universal_business.domain.customers.ports import

[CustomerRepository] - solo los puertos que realmente necesite.

D.6 `infrastructure/`, `api/`, `verticals/`

Solo [__init__.py] cada uno. Sin implementación alguna en 0.1. Marcan límites arquitectónicos y permiten que los tests de arquitectura detecten imports inversos.

D.7 BaseEntity / AggregateRoot (justificación de herencia - Corrección 12)

Se mantienen las dos clases base porque aportan valor real, no cosmético:

- Identidad compartida: Todas las entidades tienen PK UUID + igualdad por ID.
- Timestamps consistentes: [created_at] y [updated_at] UTC-aware se gestionan en un solo lugar, evitando que 10 módulos lo implementen de 10 maneras distintas.
- Mecanismo de eventos agregado: [AggregateRoot.events()] es el único punto de salida de eventos de dominio; esencial para el patrón outbox (ADR-004 / FASE 3).
- Comparte validación: [__post_init__] central puede validar tz-aware en timestamps.

No hay [BaseAuditable], [BaseSoftDelete], [BaseVersioned] ni otras jerarquías prematuras.

E. Plan de ADRs (ADR-001 a ADR-007) - Corrección 1 aplicada

> Formato estándar de cada ADR: Contexto -> Alternativas consideradas -> Decisión -> Consecuencias (positivas / negativas / mitigaciones).

ADR-001: Monolito modular vs Microservicios

(sin cambios respecto v1, pero se añade sección "Alternativas consideradas" para cumplir Corrección 1)

Sección	Contenido
Contexto	El roadmap recomienda monolito modular inicial. ¿Por qué no microservicios desde el día 1?
Alternativas consideradas	A) Microservicios desde cero (1 servicio por módulo: 12 servicios + API Gateway + service mesh); B) Modular monolith con strict boundaries + architecture tests; C) Single-tier sin límites.
Decisión	Opción B. Migrar a microservicios SOLO cuando: (a) 2+ verticales en producción, (b) límites validados empíricamente, (c) necesidad operacional demostrada.
Consecuencias	[= plan v1] Despliegue único; límites por paquete; riesgo de monolito distribuido mitigado por tests arquitectónicos.

ADR-002: Estrategia multi-tenant y aislamiento

(actualizado según Corrección 4 + Corrección 10: Tenant NO es necesariamente entidad legal; tabla matriz tenancy)

Sección	Contenido
Contexto	Roadmap requiere multi-tenant from-day-one. Jerarquía [Tenant -> Business -> Location]. Hay que definir: qué entidades llevan cada ID; cuándo es obligatorio/opcional.
Alternativas consideradas	A) Aislamiento físico por DB/schema; B) Lógico por columna + redundancia; C) Híbrido.
Decisión	Opción B + redefinición de Tenant = límite SaaS, no necesariamente persona jurídica. Matriz de tenancy al final de este ADR. Redundancia intencional de [tenant_id] en todas las entidades operacionales (incluidas las subordinadas) para evitar JOINS en queries de aislamiento.
Consecuencias	Baja complejidad operacional; filtros obligatorios; una sola DB para todos los tenants.

Matriz de tenancy ADR-002 (Corrección 10):

Entidad	[tenant_id]	[business_id]	[location_id]	Notas
Tenant	PK (si mismo)	N/A	N/A	Raíz del aislamiento.
Business	? OBLIGATORIO	PK (si mismo)	N/A	Business siempre pertenece a 1 Tenant.
Location	? OBLIGATORIO (redundante)	? OBLIGATORIO	PK (si mismo)	Invariante: [location.tenant_id == location.business.tenant_id].
Customer	? OBLIGATORIO	? OBLIGATORIO	?? OPCIONAL (NO parte de identidad)	Corrección 3: un customer se relaciona con múltiples locations del business sin duplicarse.
CatalogItem (FASE 1)	? OBLIGATORIO	? OBLIGATORIO	?? OPCIONAL	Si opcional -> catálogo de business entero.
Resource (FASE 1)	? OBLIGATORIO	? OBLIGATORIO	? OBLIGATORIO	Un Resource está en una Location física (mesa, sala, profesional).
Reservation (FASE 1)	? OBLIGATORIO	? OBLIGATORIO	? OBLIGATORIO	La reserva es siempre en una Location concreta.
Order (FASE 1)	? OBLIGATORIO	? OBLIGATORIO	? OBLIGATORIO	El pedido se factura y/o entrega en una Location.
Fulfillment (FASE 1)	? OBLIGATORIO	? OBLIGATORIO	? OBLIGATORIO	Cumplimiento ligado a Location origen.
DomainEvent	?? OPCIONAL (si aplica)	?? OPCIONAL	?? OPCIONAL	Eventos transversales sin tenancy permitidos (ej: system-level).

ADR-003: Modelo Product / Service / CatalogItem

(= v1, añadida sección "Alternativas consideradas")

Sección	Contenido
Contexto	Core agnóstico debe representar productos físicos, servicios, composiciones.
Alternativas consideradas	A) [Product] y [Service] como entidades separadas (duplicación); B) [CatalogItem] superclase con [type] discriminator; C) Tabla por tipo + JOIN.
Decisión	Opción B. [CatalogItem.type ? {PRODUCT, SERVICE, BUNDLE, DIGITAL}], [Variant] con SKU, [ModifierGroup/ModifierOption]. Nombres prohibidos: PiezaDePollo, ComboPicaPollo, Yuca, MesaRestaurante.
Consecuencias	Ambas verticales (pica / peluquería) se representan sin tocar core. Riesgo sobre-abstracción mitigado con BusinessSettings.

ADR-004: Eventos de dominio y patrón Outbox

(actualizado para Corrección 13: evento base mínimo; NO implementar dispatcher/outbox físico)

Sección	Contenido
Contexto	Desacoplar auditoría, notificaciones e integraciones.
Alternativas consideradas	A) Llamadas directas a notifier/repository dentro de aggregate methods (acoplamiento); B) Aggregate recolecta DomainEvents + aplicación persiste en outbox TX única; C) Event bus transaccional.
Decisión	Opción B pero faseada: (0.1) solo [DomainEvent] base + [AggregateRoot.event_collection]. (FASE 2/3) implementar tabla outbox + dispatcher. En ningún caso el domain envía eventos directamente.
Consecuencias	Consistencia transaccional (cuando se implemente). Complejidad futura planificada.

ADR-005: Modelo de extensiones verticales (Regla de Oro)

(= v1, sección "Alternativas consideradas" añadida)

Sección	Contenido
Contexto	Pica-pollo NO contamina core. Cómo extender sin romper.
Alternativas consideradas	A) Monkeypatching; B) Herencia en core para cada vertical; C) 3 niveles en orden: Configuración -> Datos semilla -> Extensiones específicas de vertical.
Decisión	Opción C + dirección de dependencias solo vertical -> application -> domain (tests arquitectónicos verifican imports inversos).
Consecuencias	Nuevos verticales son carpetas nuevas. "Feature envy" detectado por test de nombres prohibidos.

ADR-006: Representación de Recursos y Disponibilidad - NUEVO (Corrección 1)

Sección	Contenido
Contexto	PDF define [Resources] como "mesa, profesional, sala, pista, equipo" y [Availability] como "calendarios, capacidad, ventanas, bloqueos y reglas". Ambos conceptos se usan en múltiples verticales: restaurante (mesa / horario apertura), peluquería (profesional / turnos), clínica (sala / agenda).
Alternativas consideradas	A) [Resource] genérico + [AvailabilityRule] separado; B) [Availability] mergeado dentro de [Resource] (1:1); C) [Schedule] entidad separada con relación N:M a Resource.

Decisión	Opción A (detallado a implementar en FASE 1): (a) [Resource] = cosa reservable atómica con [ResourceType] (TABLE, ROOM, STAFF, EQUIPMENT, SLOT, OTHER...) y [location_id] OBLIGATORIO (un Resource pertenece físicamente a una Location); (b) [AvailabilityRule] = entidad separada con [effective_range: DateRange], [time_ranges: list[TimeRange]], [capacity: int?], [recurrence], [priority], [resource_id?] (para 1 recurso) o [resource_type?] (para todos del mismo tipo) o [location_id] (global); (c) [Block] = excepción explícita a reglas (feriado, baja, mantenimiento). El motor de disponibilidad (FASE 1) evalúa reglas + blocks para un [(resource_id or resource_type, location_id, date_range_or_time_range)] y retorna slots disponibles + capacidad residual. En 0.1: solo contrato [IResourceRepository] + [IAvailabilityRepository].
----------	--

Consecuencias positivas	Una misma Location puede tener Resources múltiples (pica-pollo: mesas; peluquería: sillas + profesionales); reglas compartidas vs. específicas bien diferenciadas.
-------------------------	--

Consecuencias negativas / mitigaciones	Modelo potencialmente complejo -> mitigado por ADR con decisiones tomadas a priori; motor de disponibilidad unit-testable sin infraestructura.
--	--

ADR-007: Idempotencia para Pedidos y Reservas - NUEVO (Corrección 1)

Sección	Contenido
Contexto	PDF establece: "Las operaciones externas sensibles utilizarán idempotencia para evitar pedidos o reservas duplicados." Pedidos/reservas son las operaciones más sensibles; un reintento de cliente (o un webhook reenviado) NO debe duplicarlas.
Alternativas consideradas	A) Idempotency-Key en capa HTTP + tabla [idempotency_keys] por endpoint; B) [IdempotencyKey] entidad de dominio con [(scope, key)] único, donde [scope ? {ORDER_CREATE, RESERVATION_CREATE, ORDER_CONFIRM, RESERVATION_CONFIRM...}]; C) LLaves por customer [external_ref] + fecha (débil, colisiones).
Decisión	Opción B. [IdempotencyKey] es un value object/entidad compartida (FASE 1) con: [idempotency_id: UUID], [scope: IdempotencyScope enum], [key: str], [aggregate_type], [aggregate_id?], [created_at], [locked_until?] (TTL). Contrato estricto: (a) Los casos de uso [create_order] / [create_reservation] (FASE 1) REQUIEREN un [IdempotencyKey] como parámetro; (b) Si la llave ya existe -> retornar el aggregate ya creado (sin repetir???); (c) Si la llave NO existe -> crear aggregate + registrar llave con su aggregate_id en la MISMA transacción. Exclusión a nivel [(scope, key, tenant_id, business_id)] (único por tenant/business, no global). En 0.1: NO se implementa la entidad [IdempotencyKey], pero SÍ se define su contrato y se reserva espacio en [domain/shared/] (el test de arquitectura lo puede referenciar por nombre sin que exista el archivo, o crear el VO con el mínimo). Dejar explícito que application services (FASE 2) lo validarán.
Consecuencias positivas	Reintentos seguros por diseño; trazabilidad de operaciones; integración con webhooks.

Consecuencias negativas / mitigaciones

Toda API de mutación requiere llave -> mitigado con valor por defecto (hash determinista) en canales que no la proveen; TTL configurables por scope para evitar crecimiento infinito de la tabla.

F. Diseño de Value Objects (revisado)

F.1 IDs (`domain/shared/value\_objects/ids.py`)

= plan v1 pero explícitamente wrapper dataclass (no NewType), porque aporta seguridad runtime.

```
# Pseudocódigo
@dataclass(frozen=True)
class EntityId(Generic[T]):
    value: uuid.UUID
    @classmethod
    def new(cls) -> EntityId[T]: ...
    def __post_init__(self) -> None:
        if not isinstance(self.value, uuid.UUID):
            raise InvariantViolation("EntityId.value debe ser UUID")

# Idéntica estructura por tipo:
@dataclass(frozen=True)
class TenantId:    value: uuid.UUID ...
@dataclass(frozen=True)
class BusinessId:  value: uuid.UUID ...
@dataclass(frozen=True)
class LocationId:  value: uuid.UUID ...
@dataclass(frozen=True)
class CustomerId:  value: uuid.UUID ...
```

F.2 Money (`domain/shared/value\_objects/money.py`) - CORREGIDO (Corrección 5)

Decision final, políticamente explícita:

```

# Pseudocódigo
from decimal import Decimal, ROUND_HALF_EVEN, getcontext
from dataclasses import dataclass

# ----- POLÍTICA (decidida aquí, NO configurable por instancia) -----
MONEY_PRECISION: Final[int] = 10          # precision interna Decimal
MONEY_SCALE: Final[int] = 4               # 4 decimales -> cubre impuestos hasta 0.01%
MONEY_ROUNDING: Final[str] = ROUND_HALF_EVEN # "Banker's rounding"
MONEY_ALLOWED_CURRENCIES: Final[set[str]] = {"DOP", "USD", "EUR"} # ampliar según necesidades
# -----

getcontext().prec = max(MONEY_PRECISION, getcontext().prec)

CurrencyStr = Annotated[str, "ISO-4217 3-letter uppercase"]

def _to_decimal(value: int | Decimal | str) -> Decimal:
    """Coerción SEGURA: int, Decimal o str (NO float)."""
    if isinstance(value, float):
        raise MoneyCurrencyMismatchError("Money NO acepta float. Usa Decimal(str(x)) o int.")
    if isinstance(value, int):
        return Decimal(value)
    if isinstance(value, Decimal):
        return value
    if isinstance(value, str):
        try:
            d = Decimal(value)
        except Exception as e:
            raise MoneyRoundingError(f"Money.amount str no es decimal: {value!r}") from e
        return d
    raise TypeError(f"Money.amount tipo no soportado: {type(value).__name__}")

def _validate_currency(cur: str) -> str:
    if not isinstance(cur, str) or len(cur) != 3 or not cur.isalpha():
        raise MoneyCurrencyMismatchError(f"Moneda inválida: {cur!r}")
    cur = cur.upper()
    if MONEY_ALLOWED_CURRENCIES and cur not in MONEY_ALLOWED_CURRENCIES:
        raise MoneyCurrencyMismatchError(f"Moneda no permitida: {cur!r} (activarla en MONEY_ALLOWED_CURRENCIES)")
    return cur

@dataclass(frozen=True)
class Money:
    amount: Decimal
    currency: CurrencyStr

# ----- Constructor con validación estricta -----
def __init__(self, amount: int | Decimal | str, currency: str) -> None:
    amount_dec = _to_decimal(amount)
    cur_ok = _validate_currency(currency)
    # Cuantización según escala (invariante: mismo número de decimales)
    quantized = amount_dec.quantize(Decimal(f"1E-{MONEY_SCALE}"), rounding=MONEY_ROUNDING)
    object.__setattr__(self, "amount", quantized)
    object.__setattr__(self, "currency", cur_ok)

```

```
# ----- Operaciones: solo int + Decimal -----
def _same_currency_or_fail(self, other: "Money") -> None:
    if self.currency != other.currency:
        raise MoneyCurrencyMismatchError(
            f"Mezcla de monedas: {self.currency} + {other.currency}"
        )

def add(self, other: "Money") -> "Money":
    self._same_currency_or_fail(other)
    return Money(self.amount + other.amount, self.currency)

def subtract(self, other: "Money") -> "Money":
    self._same_currency_or_fail(other)
    return Money(self.amount - other.amount, self.currency)

def multiply(self, factor: int | Decimal | str) -> "Money":
    """Precio x cantidad, impuestos %, descuentos %, tarifas, factores proporcionales."""
    factor_dec = _to_decimal(factor) # NO float
    return Money(self.amount * factor_dec, self.currency) # __init__ re-cuantiza

def divide(self, divisor: int | Decimal | str) -> "Money":
    """División proporcional (p. ej. repartir gastos)."""
    d = _to_decimal(divisor)
    if d == Decimal("0"):
        raise MoneyRoundingError("Money.divide por cero")
    return Money(self.amount / d, self.currency)

# ----- Helpers -----
@classmethod
def zero(cls, currency: str) -> "Money": ...
@property
def is_positive(self) -> bool: ...
@property
def is_negative(self) -> bool: ...
# Comparaciones: ==, <, <=, >, >= (mismo currency o falla)
```

Política de mezcla de monedas:

- Operaciones binarias [add/subtract] entre distinta moneda -> [MoneyCurrencyMismatchError] SIEMPRE.
- No hay "auto-conversión" en el dominio (no sabemos el tipo de cambio). Si alguien necesita convertir, debe hacerlo fuera con un servicio ([CurrencyConverter]) -> será un [application service] con un puerto [ICurrencyRateProvider].

Redondeo y precisión:

- Internamente [Decimal] con [prec=10].
- Todos los [Money] se cuantizan a 4 decimales (escala 4) con [ROUND_HALF_EVEN]. Esto permite impuestos/porcentajes y al finalizar el pedido se redondea a 2 decimales para el total mostrado (aplicación, no dominio).
- Errores: [MoneyRoundingError] (redondeo inseguro si p.ej. [amount=Decimal("1.12345")]) sin indicar comportamiento -> con [__init__] cuantizando automáticamente NO hay error, solo redondeo determinista).

F.3 Temporal (`domain/shared/value\_objects/temporal.py`)

= v1 pero con [require_aware] más estricto (lanza [TemporalRangeError], no [ValueError] genérico):

```
def require_aware(dt: dt.datetime) -> dt.datetime:
    """Valida que dt sea timezone-aware. Lanza TemporalRangeError si no.
    NINGÚN datetime tz=None cruza al dominio sin esta validación (Corrección 11)."""
    if dt.tzinfo is None or dt.tzinfo.utcoffset(dt) is None:
        raise TemporalRangeError(
            f"datetime debe ser timezone-aware (tzinfo no None). Valor recibido: {dt!r}"
        )
    return dt
```

F.4 Status y transiciones - CORREGIDO (Corrección 2)

> NO [LifecycleStatus] universal. Cada módulo define SU propio [XxxStatus(str, Enum)]. > Sí hay utilidades compartidas de validación.

```
# domain/shared/value_objects/status.py
"""Utilidades COMPARTIDAS para máquinas de estado finitas.
NO contiene enumeraciones de estados concretos (Corrección 2)."""
from __future__ import annotations
from dataclasses import dataclass
from enum import Enum
from typing import Generic, TypeVar
from universal_business.domain.shared.errors import StatusTransitionError

S = TypeVar("S", bound=Enum)

StatusTransitions[S] = dict[S, set[S]]

@dataclass(frozen=True)
class StatusTransition(Generic[S]):
    """Helper inmutable: matriz de transiciones permitidas por agregado."""
    valid_transitions: StatusTransitions[S]

    def can(self, from_: S, to: S) -> bool:
        return to in self.valid_transitions.get(from_, set())

    def ensure(self, from_: S, to: S) -> None:
        """Lanza StatusTransitionError con detalle si la transición no está permitida."""
        if not self.can(from_, to):
            allowed = ", ".join(sorted(x.value for x in self.valid_transitions.get(from_, set())))
            or "<ninguna>"
            raise StatusTransitionError(
                f"Transición de estado inválida: {from_.value} -> {to.value}. "
                f"Valores permitidos desde {from_.value}: {allowed}."
            )

    def transition_guard(current: S, target: S, matrix: StatusTransitions[S]) -> None:
        """Alias funcional rápido."""
        StatusTransition(matrix).ensure(current, target)
```

Ejemplos de estados POR MÓDULO (cada uno en su value_objects.py propio):

```
# domain/customers/value_objects.py
class CustomerStatus(str, Enum):
    DRAFT = "draft"
    ACTIVE = "active"
    SUSPENDED = "suspended"
    ANONYMIZED = "anonymized"
    ARCHIVED = "archived"

# domain/business/value_objects.py (TenantStatus, BusinessStatus, LocationStatus)
class TenantStatus(str, Enum):
    PENDING_ONBOARDING = "pending_onboarding"
    ACTIVE = "active"
    SUSPENDED = "suspended"
    BILLED_ONLY = "billed_only"
    TERMINATED = "terminated"

class BusinessStatus(str, Enum):
    DRAFT = "draft"
    OPERATIONAL = "operational"
    TEMPORARILY_CLOSED = "temporarily_closed"
    PERMANENTLY_CLOSED = "permanently_closed"

class LocationStatus(str, Enum):
    DRAFT = "draft"
    OPEN = "open"
    TEMPORARILY_CLOSED = "temporarily_closed"
    CLOSED = "closed"

# domain/reservations/value_objects.py
class ReservationStatus(str, Enum):
    DRAFT = "draft"
    CONFIRMED = "confirmed"
    CHECKED_IN = "checked_in"
    COMPLETED = "completed"
    CANCELLED = "cancelled"
    NO_SHOW = "no_show"

# domain/orders/value_objects.py
class OrderStatus(str, Enum):
    DRAFT = "draft"
    VALIDATED = "validated"
    CONFIRMED = "confirmed"
    PREPARING = "preparing"
    READY = "ready"
    DELIVERING = "delivering"
    COMPLETED = "completed"
    CANCELLED = "cancelled"
    REFUNDED = "refunded"

# domain/fulfillment/value_objects.py
class FulfillmentStatus(str, Enum):
    PENDING = "pending"
    ASSIGNED = "assigned"
```

```
IN_PROGRESS = "in_progress"
COMPLETED = "completed"
FAILED = "failed"
CANCELLED = "cancelled"
```

Cada módulo define además su propia matriz de transiciones [StatusTransition[*Status]] (no compartida).

G. Diseño de Entidades (core, revisado)

G.1 Jerarquía y clases base (sin cambios, justificación en D.7)

```
BaseEntity[T]
??? id: EntityId[T]  (o ID específico: TenantId...)
??? created_at: datetime  (UTC aware)
??? updated_at: datetime  (UTC aware, >= created_at)
??? __eq__ / __hash__ por id

AggregateRoot[T] (extends BaseEntity)
??? _events: list[DomainEvent]
??? record_event(event)
??? events() -> list[DomainEvent]  # copia defensiva
??? clear_events()
```

G.2 Tenant - Redefinido CORREGIDO (Corrección 4: NO necesariamente entidad legal)

Campo	Tipo	Obligatorio?	Invariantes / Notas
[id]	[TenantId]	?	PK UUID v4.
[display_name]	[str]	?	2-100 chars (nombre visible en UI / facturas / tenants list).

[legal_entity_name]	`str \	None`	?? OPCIONAL	Nombre legal de la persona jurídica. Si [None] -> Tenant NO es una entidad legal (p. ej.: una cuenta SaaS individual, una organización informal). Corrección 4.
---------------------	--------	-------	-------------	---

[legal_tax_id]	`str \	None`	?? OPCIONAL	NIF/RNC/CUIT/... Solo si [legal_entity_name] existe.
[status]	[TenantStatus] (enum PROPIO, NO LifecycleStatus)	?	PENDING_ONBOARDING -> ACTIVE -> SUSPENDED -> {BILLED_ONLY / TERMINATED}.	
[created_at] / [updated_at]	UTC aware	?	heredado de BaseEntity.	

Definición canónica: > *Tenant es el límite superior de aislamiento, propiedad y separación de datos dentro de la plataforma SaaS.*

Puede representar indistintamente: empresa, grupo empresarial, franquicia, cuenta de cliente SaaS, organización con varias unidades operativas. NO es sinónimo de "persona jurídica".

G.3 Business - Subordinado a Tenant (sin cambios de campos pero sí de semántica)

Campo	Tipo	Invariantes
[id]	[BusinessId]	PK.
[tenant_id]	[TenantId]	? OBLIGATORIO (multi-tenant).
[name]	[str]	2-100 chars.
[description]	`str`	None`<= 1000 chars.
[contact_info]	[ContactInfo]	Email o teléfono al menos uno presente.
[settings]	[BusinessSettings]	[default_currency: CurrencyStr], [feature_flags: dict[str, bool]] (flags mínimos, sin nombres verticales).
[status]	[BusinessStatus] (propio enum)	
[created_at] / [updated_at]	UTC aware	

G.4 Location - Tercer nivel bajo Business

Campo	Tipo	Obligatorio	Invariantes
[id]	[LocationId]	?	PK.
[tenant_id]	[TenantId]	?	Redundante intencional (isolation sin JOIN). Invariante: [location.tenant_id == location.business.tenant_id] (Factory valida).
[business_id]	[BusinessId]	?	
[name]	[str]	?	2-100 chars.
[address]	[Address]	?	[country] ISO-3166 alpha-2 obligatorio.
[timezone]	[str] (IANA)	?	Ej. ["America/Santo_Domingo"]. TODAS las reglas de disponibilidad se evalúan en esta zona.
[operating_hours]	[OperatingHours]	?	7 días. Cada día = 0..N [TimeRange] (0 = cerrado).
[contact_info]	`ContactInfo`	None`	?? Si None -> property hereda [Business.contact_info].
[status]	[LocationStatus] (propio)	?	
[created_at] / [updated_at]	UTC aware	?	

G.5 Customer - CORREGIDO (Corrección 3: location_id NO obligatorio, NO identidad)

Campo	Tipo	Obligatorio	Invariantes / Notas	
[id]	[CustomerId]	?	PK.	
[tenant_id]	[TenantId]	? OBLIGATORIO	Corrección 10. Cliente pertenece a un tenant.	
[business_id]	[BusinessId]	? OBLIGATORIO	Corrección 3: customer existe dentro de un business.	
[location_id]	`LocationId`	None`	?? OPCIONAL	NO forma parte de la identidad del customer. Puede ser [None] (el customer es del business entero, visitará locations distintas, se relaciona con varias por medio de Orders/Reservations). Operations como Order/Reservation Sí traerán [location_id] concreta.
[external_ref]	`str`	None`	??	ID sistema origen (WhatsApp JID, CRM ID, ...). Único por [(tenant_id, business_id, external_ref)] cuando no es None.
[given_name]	[str]	?	1-100 chars.	
[family_name]	`str`	None`	??	1-100 chars.
[full_name]	[property str]	-	[(given_name + " " + family_name).strip()].	
[contact_points]	[list[ContactPoint]]	?	Para [ACTIVE] debe haber len >= 1. DRAFT permite 0.	

[addresses]	[list[Address]]	?	0..N.
[consents]	[list[Consent]]	?	
[preferences]	[CustomerPreferences]	?	
[status]	[CustomerStatus] (propio)	?	DRAFT -> ACTIVE -> {SUSPENDED / ANONYMIZED / ARCHIVED}.
[created_at] / [updated_at]	UTC aware	?	

Tenancy resumida Customer (Corrección 10):

```
Customer.tenant_id : OBLIGATORIO (aislamiento SaaS)
Customer.business_id: OBLIGATORIO (unidad operativa)
Customer.location_id: OPCIONAL (NO parte de identidad; puede ser None)
```

H. Diseño de Puertos / Interfaces de Repositorio - CORREGIDO (Corrección 7)

H.1 Principios generales

1. Puertos CERCA de su dominio: [domain/<modulo>/ports.py]. Correcto: [domain/customers/ports.py] define [ICustomerRepository].
2. NO punto central agregador: Eliminar [application/ports/repositories.py] (acoplador).
3. [typing.Protocol] (no [abc.ABC]) - duck-typing estático + fakes en tests sin herencia.
4. Todas las consultas tienen contexto de tenancy. Ejemplo: NO existe [list()], sí [list_by_tenant(tenant_id)] o [list_by_business(business_id)].
5. Retornan entidades de dominio (no dicts, no rows).
6. Ningún import de SQLAlchemy/FastAPI. Architecture Test A-4/A-5 garantiza esto.

H.2 Interfaces por módulo (ejemplos mínimos)

```
# domain/business/ports.py
class ITenantRepository(Protocol):
    def get(self, tenant_id: TenantId) -> Tenant | None: ...
    def list(self, *, status: TenantStatus | None = None) -> list[Tenant]: ...

class IBusinessRepository(Protocol):
    def get(self, business_id: BusinessId) -> Business | None: ...
    def list_by_tenant(self, tenant_id: TenantId,
                      *, status: BusinessStatus | None = None) -> list[Business]: ...

class ILocationRepository(Protocol):
    def get(self, location_id: LocationId) -> Location | None: ...
    def list_by_business(self, business_id: BusinessId,
                        *, status: LocationStatus | None = None) -> list[Location]: ...
```

```
# domain/customers/ports.py
class ICustomerRepository(Protocol):
    def get(self, customer_id: CustomerId) -> Customer | None: ...
    def get_by_external_ref(
        self,
        *,
        tenant_id: TenantId,
        business_id: BusinessId,
        external_ref: str,
    ) -> Customer | None: ...
    def search(
        self,
        *,
        tenant_id: TenantId,
        business_id: BusinessId,
        location_id: LocationId | None = None, # None = all locations del business
        query: str,
    ) -> list[Customer]: ...
```

```
# 6 módulos restantes - contratos MÍNIMOS (FASE 1 ampliará métodos):
# domain/catalog/ports.py          -> ICatalogRepository.get(item_id, tenant_id, business_id)
# domain/resources/ports.py        -> IResourceRepository.get(resource_id, tenant_id, location_id)
# domain/availability/ports.py     -> IAvailabilityRepository.list_for(resource_id, date_range)
# domain/reservations/ports.py    -> IReservationRepository.get(reservation_id, tenant_id)
# domain/orders/ports.py          -> IOrderRepository.get(order_id, tenant_id)
# domain/fulfillment/ports.py     -> IFulfillmentRepository.get(fulfillment_id, tenant_id)
```

I. Diseño de DomainEvent base - CORREGIDO (Corrección 13)

> Mínimo útil. Sin dispatcher ni tabla outbox físico (FASE 3).

```
# domain/shared/events.py
"""DomainEvent base mínimo útil (Corrección 13).
Sirve para auditoría, notificaciones, integraciones y patrón outbox (futuro)."""

from __future__ import annotations
from dataclasses import dataclass, field
from datetime import datetime, timezone
from typing import Any, ClassVar, Optional
import uuid

def _utc_now() -> datetime:
    return datetime.now(tz=timezone.utc)

@dataclass(frozen=True)
class DomainEvent:
    # ===== OBLIGATORIOS (Corrección 13: siempre presentes) =====
    event_id: uuid.UUID = field(default_factory=uuid.uuid4)
    occurred_at: datetime = field(default_factory=_utc_now)
    aggregate_id: str          # str(uuid) - serializable, siempre
    aggregate_type: str        # "Tenant", "Customer", "Order", "Reservation", ...

    # ===== OPCIONALES, solo cuando el agregado correspondiente los tiene =====
    tenant_id: Optional[str] = None
    business_id: Optional[str] = None
    location_id: Optional[str] = None
    metadata: dict[str, Any] = field(default_factory=dict) # correlation_id, causation_id, actor,
    source...

    # ===== Sobre-escribir en subclases =====
    event_type: ClassVar[str] = "domain.event"

    def __post_init__(self) -> None:
        # Invariante 1: occurred_at es siempre UTC-aware
        if self.occurred_at.tzinfo is None or self.occurred_at.tzinfo.utcoffset(self.occurred_at) is
None:
            raise InvariantViolation("DomainEvent.occurred_at debe ser timezone-aware (UTC
recomendado)")
        # Invariante 2: aggregate_id y aggregate_type nunca vacíos
        if not self.aggregate_id:
            raise InvariantViolation("DomainEvent.aggregate_id es obligatorio")
        if not self.aggregate_type:
            raise InvariantViolation("DomainEvent.aggregate_type es obligatorio")
```

Subclases a implementar en FASE 1/2 (no en 0.1):

- [TenantCreated / TenantStatusChanged]
- [BusinessCreated / BusinessSettingsUpdated]
- [LocationOperatingHoursUpdated]
- [CustomerCreated / CustomerContactPointVerified / CustomerConsentUpdated]
- [OrderCreated / OrderStatusChanged / OrderPaymentApplied]
- [ReservationCreated / ReservationStatusChanged]
- etc.

J. Estrategia de Tests - CORREGIDA según Corrección 14

J.1 Estructura de carpetas actualizada

```
tests/
??? architecture/          # <- B. Tests arquitectónicos
?   ??? test_architecture_boundaries.py
??? imports/              # <- C. Tests de importación
?   ??? test_import_without externals.py
??? unit/domain/          # <- A. Tests unitarios
    ??? shared/
    ?   ??? test_ids.py
    ?   ??? test_money.py
    ?   ??? test_temporal.py
    ?   ??? test_status_transition.py  # NO LifecycleStatus
    ?   ??? test_domain_events.py
    ?   ??? test_base_entities.py     # BaseEntity/AggregateRoot valor real
??? business/
    ?   ??? test_tenant.py            # Tenant NO necesariamente entidad legal
    ?   ??? test_business.py
    ?   ??? test_location.py
??? customers/
    ??? test_customer.py             # location_id NO obligatorio
```

J.2 A. Tests unitarios (Corrección 14.A)

Archivo	Qué prueba	Nº tests aprox
[test_ids.py]	Generación UUID v4 válida; invalid inputs -> error; igualdad por valor; [TenantId] != [BusinessId] con mismo UUID interno; [__repr__] util.	7
[test_money.py]	Constructor RECHAZA [float] (runtime TypeError); int, Decimal, str aceptados; moneda minúscula -> normalizada upper; moneda no permitida -> error; [add()] misma moneda OK; [add()] distinta moneda -> [MoneyCurrencyMismatchError]; [subtract()] resultado negativo permitido (ej: descuentos); [multiply(int=3)] OK; [multiply(Decimal("0.18"))] OK (impuesto 18%); [multiply(Decimal("0.85"))] OK (15% dto); [multiply(float)] -> TypeError; [divide(int)] OK; [divide(Decimal("0"))] -> [MoneyRoundingError]; comparaciones [< > <= >= ==]; moneda distinta en comparación -> falla; cuantización a 4 decimales (p. ej. [Decimal("1.12345")] -> ["1.1235"] con ROUND_HALF_EVEN).	22

[test_temporal.py]	[require_aware(datetime_naive)] -> 11 [TemporalRangeError]; [require_aware(utc)] OK; [DateRange] end<start -> error; [TimeRange] distinta tz -> error; [TimeRange] ingenuo -> error; [overlap(a,b)] 4 casos: anterior, posterior, intersección estricta, touching (contigüidad no overlap); [as_utc] convierte correctamente DST.
--------------------	---

[test_status_transition.py]	[StatusTransition[CustomerStatus]] 6 DRAFT->ACTIVE pasa; ACTIVE->DRAFT falla con mensaje que lista permitidos; [transition_guard] alias; 2 matrices distintas (Customer vs Order) comparten StatusTransition sin interferir; terminal status (ARCHIVED) no transiciona a nada.
-----------------------------	--

[test_domain_events.py]	Evento sin tz -> [InvariantViolation]; 5 aggregate_id/type vacíos -> error; [record_event()/events()] retorna copia (mutar la copia NO afecta al agregado); [clear_events()] limpia; eventos con/sin tenancy context.
-------------------------	--

[test_base_entities.py]	[BaseEntity] igualdad por id (datos distintos = mismo id -> [__eq__ True]); created_at/updated_at UTC-aware al nacer; updated_at >= created_at tras mutación (actualizar [name] -> [touch()]) actualiza updated_at; [AggregateRoot.events()] copia defensiva (párrafo anterior).	5
[test_tenant.py]	Constructor happy path; [legal_entity_name = None] (caso tenant sin personalidad jurídica) funciona; status PENDING_ONBOARDING->ACTIVE pasa; ACTIVE->PENDING_ONBOARDING falla.	4
[test_business.py]	Constructor happy path; contact_info sin email ni phone -> error; settings.default_currency "PE" -> error.	3
[test_location.py]	Constructor happy path; timezone IANA ["America/Santo_Domingo"] pasa; timezone ["Foo/Bar"] -> error; address.country ["DO"] pasa, ["D"] (longitud !=2) -> error; Factory con [tenant_id] que no coincide con business.tenant_id -> [TenantBoundaryViolationError].	5
[test_customer.py]	Constructor happy path con [location_id=None] pasa (caso principal Corrección 3); Constructor con location_id válido también pasa (no es excluyente); Transición DRAFT->ACTIVE con contact_points=[] -> [StatusTransitionError].	4

Total tests unitarios 0.1: ~= 72 tests.

J.3 B. Tests arquitectónicos (Corrección 14.B) - archivo único

[tests/architecture/test_architecture_boundaries.py] - casos:

ID	Regla (Corrección 9)	Cómo se verifica
AT-1	domain NO importa infrastructure	Inspección de [importlib] / [sys.modules]: importar todos los submódulos de [domain]; comprobar que [universal_business.infrastructure] NO aparece en los [module.__file__] de [sys.modules] ni en AST de imports.
AT-2	domain NO importa api	Análogo AT-1 para [universal_business.api].
AT-3	domain NO importa verticals	Análogo AT-1 para [universal_business.verticals].
AT-4	domain NO depende de FastAPI	Después de importar domain, NINGUNA key de [sys.modules] empieza por [fastapi] / [starlette].
AT-5	domain NO depende de SQLAlchemy	Análogo AT-4: [sqlalchemy], [psycopg], [asyncpg], [alembic], [sqlite3] NO en [sys.modules].
AT-6	Core no contiene nombres específicos de pica pollo	Grep case-insensitive sobre todos los [.py] de [src/universal_business/] (menos [verticals/], pero en 0.1 [verticals/] está vacío). Patrones prohibidos: [pica.?pollo], [piccapollo], [pollo], [restaurant.*mesa], [mesarestaurante], [repartidorrestaurante], [yuca], [platano], [salon(?![a-z])], [clinic]. Si alguno aparece -> test FAIL.
AT-7	verticals NO se filtran dentro del core (carga circular)	[domain] y [application] NO tienen imports que incluyan la cadena [verticals]. AST walk.

AT-8	Dependencia correcta direction check	[infrastructure.__init__] importar de domain OK; [api.__init__] importar de application OK; domain no importar de application. [importlib] en orden.
------	--------------------------------------	--

J.4 C. Tests de importación (Corrección 14.C)

[tests/imports/test_import_without externals.py] - caso único pero exhaustivo:

```
# Pseudocódigo del test:
# 1. sys.modules = {} (reset simulado con subprocess aislado PREFERIBLEMENTE,
#    o usando importlib para importar en orden y comprobar).
# 2. pip list / pkgutil: antes de importar universal_business, marcar los
#    módulos ya cargados.
# 3. import universal_business.domain
# 4. Assert: fastapi, sqlalchemy, psycpg, alembic, pywhatkit, twilio,
#    firebase_admin, openai, stripe, googlemaps NO aparecen.
# 5. Assert: TODOS los módulos .business, .customers, .catalog, .resources,
#    .availability, .reservations, .orders, .fulfillment se importan
#    OK sin levantar infraestructura.
# 6. Assert: import universal_business (top level) también pasa.
```

Implementación recomendada: Usar [subprocess.run([sys.executable, "-c", ...], capture_output=True)] para aislar completamente el entorno de importación (módulo [multiprocessing] también sirve, pero subprocess es más simple).

K. Estrategia de Tipado Estático / Linting - CORREGIDO (Corrección 15: mínima y coherente)

K.1 Stack (SOLO pytest + ruff + mypy. SIN bandit SIN import-linter SIN extras)

Categoría	Herramienta	Versión mín.	Justificación
Tests	[pytest]	>=8.0	Standard.
Coverage	[pytest-cov]	>=5.0	Integrado.
Lint + isort + format	[ruff]	>=0.6	Todo en uno. Veloz.
Type checking	[mypy]	>=1.10	Estándar strict mode.

K.2 `pyproject.toml` (dependencias)

```
[project]
name = "universal-business-core"
version = "0.1.0"
description = "Universal Business Core (Entrega 0.1) - dominio agnóstico multi-tenant"
readme = "README.md"          # NOTA: si no hay README, omitir o crear mínimo.
requires-python = ">=3.11"
dependencies = [
    "tzdata; sys_platform == 'win32'",    # Windows no lleva tzdata en stdlib Py <=3.11
]

[project.optional-dependencies]
test = ["pytest>=8.0", "pytest-cov>=5.0"]
lint = ["ruff>=0.6", "mypy>=1.10"]
dev = ["pytest>=8.0", "pytest-cov>=5.0", "ruff>=0.6", "mypy>=1.10"]
```

K.3 Configuración mypy

```
[tool.mypy]
python_version = "3.11"
strict = true
warn_return_any = true
warn_unused_configs = true
explicit_package_bases = true
ignore_missing_imports = false
show_error_codes = true
mypy_path = "src:tests"
no_implicit_optional = true
disallow_subclassing_any = true

# Relajación TEMPORAL para módulos que en 0.1 solo tienen contratos:
[[tool.mypy.overrides]]
module = [
    "universal_business.domain.catalog.*",
    "universal_business.domain.resources.*",
    "universal_business.domain.availability.*",
]
strict = false
```

K.4 Configuración ruff

```
[tool.ruff]
target-version = "py311"
line-length = 99
src = ["src", "tests"]
exclude = [".venv", "build", "dist", "docs"]

[tool.ruff.lint]
select = ["E", "F", "I", "N", "UP", "B"]
ignore = ["B008"] # función en args default (dataclass field default_factory no aplica)

[tool.ruff.format]
quote-style = "double"
indent-style = "space"
```

K.5 `.pre-commit-config.yaml`

Mantenido como opcional (Corrección 15). Se documenta que es opcional; CI ejecuta las comprobaciones de todas formas.

L. Estrategia CI (`.github/workflows/ci.yml`) - CORREGIDO 3 pasos MÍNIMOS (Corrección 15)

> Pipeline MÍNIMO. SOLO: ruff check -> mypy -> pytest. Sin extras.

```
name: CI - Entrega 0.1 Architectural Baseline

on:
  push:
    branches: [ feat/architectural-baseline ]
  pull_request:
    branches: [ feat/**, master ]

jobs:
  quality:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        python-version: [ "3.11" ]

    steps:
      - uses: actions/checkout@v4

      - name: Setup Python ${ matrix.python-version }
        uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }

      - name: Instalar dependencias
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[dev,test,lint]"

      - name: 1) Ruff lint + format check
        run: |
          ruff check .
          ruff format --check .

      - name: 2) Mypy strict (shared + business + customers)
        run: |
          mypy --strict \
            src/universal_business/domain/shared \
            src/universal_business/domain/business \
            src/universal_business/domain/customers \
            tests

      - name: 3) Pytest + coverage >= 60%
        run: >
          pytest -q
          --cov=src/universal_business
          --cov-report=term
          --cov-report=xml
          --cov-fail-under=60
```


M. GATE 0.1 - Criterios de aceptación VERIFICABLES (CORREGIDO según Corrección 16, 14 puntos)

> La entrega se considera válida ÚNICAMENTE si TODOS los 14 criterios se cumplen. > Ningún criterio "parcialmente válido".

#	Criterio	Cómo se VERIFICA
G1	El paquete [universal_business] puede importarse correctamente.	[python -c "import universal_business"] exit code 0; [tests/imports/test_import_without_external.py] PASSA.
G2	TODOS los tests pasan (unit + arquitectura + importación; ~72+8+1 ~81 tests).	[pytest -q] exit code 0.
G3	Ruff no detecta errores.	[ruff check .] exit 0; [ruff format --check .] exit 0.

G4	Mypy pasa según la configuración definida.	[mypy --strict src/universal_business/domain/shared src/universal_business/domain/business src/universal_business/domain/customers tests] exit 0. Los módulos esqueleto (catalog/resources/availability) pasan con strict=false en overrides.			
G5	El dominio NO depende de FastAPI (incluyendo starlette).	Architecture Test AT-4 PASSA; `grep -r "fastapi\`	starlette" src/universal_business/domain` 0 coincidencias.		
G6	El dominio NO depende de SQLAlchemy (incl. psycopg, alembic, asyncpg, sqlite native driver imports).	Architecture Test AT-5 PASSA; `grep -rE "sqlalchemy	psycopg alembic	asyncpg" src/universal_business/domain` 0 coincidencias.	

G7	El dominio NO depende de una base de datos (no levanta conexiones, no requiere drivers).	Architecture Test AT-5 PASSA; Test de importación J.4 PASSA.
----	---	---

G8	No existen nombres ni reglas específicas del vertical pica-pollo dentro del core ([src/universal _business/], excluyendo [verticals/] que está vacío en 0.1).	Architecture Test AT-6 PASSA (grep automático con patrones).
----	--	---

G9	Tenant, Business, Location y Customer existen como conceptos de dominio reales (no placeholders). Tienen campos, invariantes y tests.	Archivos [domain/busin ess/entities.py] y [domain/custo mers/entities. py] existen y contienen las 4 entidades; [test_tenant], [test_busines s], [test_location] , [test_custome r] existen y PASSAN.
----	---	--

G10	Value objects básicos implementados y probados: IDs fuertes, Money, Currency, DateRange, TimeRange, tz-aware datetime guards.	Tests unitarios [test_ids.py], [test_money.py], [test_temporal.py] PASSAN (coverage >=90% en shared VO).				
G11	Repositorios definidos ÚNICAMENTE como contratos/portos ([Protocol] o [ABC]) CERCA de su dominio; NO dependencias de ORM; NO punto central de reexports.	Los 9 [ports.py] existen y son Protocol; Architecture Test AT-5 comprueba no SQLAlchemy imports; [application/ports/] no existe (Corrección 7).				
G12	Infrastructure, API y verticals NO contienen implementaciones prematuras. Solo [__init__.py] vacíos.	`find src/universal_business/infrastructure src/universal_business/api src/universal_business/verticals -type f -name "*.py" xargs wc -l  <= 1 línea cada uno (solo comentarios o vacío); ]grep -rE "class def import sqlalchemy from fastapi" ...` sin resultados.				

G13	Existen tests arquitectónicos que protegen los límites (8 reglas AT-1...AT-8).	Archivo [tests/architecture/test_architecture_boundedaries.py] existe; 8 tests PASSA.
-----	--	---

G14	El core puede probarse completamente sin levantar servicios externos. Todo suite pytest corre offline (no hay marcadores [skipif] condicionado a servicios externos).	Ejecutar pytest sin internet, sin variables de entorno de servicios; exit 0. Coverage report se genera sin levantar nada.
-----	---	---

N. Riesgos, ambigüedades y decisiones PENDIENTES (CORREGIDO, solo las que requieren decisión humana)

ID	Ambigüedad / Riesgo	Detalle	Decisión requiere HUMANA?	Sugerencia de mitigación / default
A1	[business_id] en Customer -> ¿OBLIGATORIO o opcional?	Corrección 3: [location_id] opcional está decidido; sobre [business_id] el plan v2 lo marca obligatorio. ¿Qué pasa si un Tenant quiere customers a nivel tenant sin business?	Sí requiere decisión si existe ese caso.	Default actual: OBLIGATORIO (la operación real siempre sucede dentro de un business). Si hay caso de uso "global customer account" -> cambiar a opcional en FASE 1.
A2	Política [MONEY_ALLOWED_CURRENCIES]: whitelist vs. cualquier ISO 4217.	Plan actual: whitelist [{"DOP","USD","EUR"}]. ¿Agregar todas las 180?	Sí/No.	Default: whitelist y documentar cómo ampliarla (1 línea en money.py).
A3	Money: 4 decimales internos -> ¿moneda [DOP] acepta 4 decimales?	Pesos RD usan 2 decimales; impuestos y factores se calculan con más.	No requiere humana si contabilidad acepta.	Default actual: 4 decimales internos + 2 para display (decisión application).

A4	Entity IDs: wrapper dataclass frozen vs. NewType.	Wrapper aporta runtime safety pero es más verboso. NewType más ligero pero sin seguridad en runtime.	No requiere si aceptamos el default plan.	Default: wrapper (más seguro). Si el equipo prefiere NewType, cambiar y añadir test [isinstance(TenantId(x), str)].
A5	BaseEntity / AggregateRoot: ¿[__post_init__] valida automáticamente que [created_at] y [updated_at] son timezone-aware?	Plan actual: Sí. ¿Si se olvida pasar tz debe fallar en construcción?	No, solo decisión de implementación.	Default: sí valida.
A6	¿Nombre de [Tenant.legal_entity_name]/[legal_tax_id] ó [legal_name]/[tax_id]?	Corrección 4 dice Tenant NO es necesariamente entidad legal. Campo debe reflejar optatividad.	Pregunta de naming (estética).	Default: `legal_entity_name: str` None[, None`.]legal_tax_id: str
A7	¿Generar [.pre-commit-config.yaml] aunque sea opcional? Corrección 15 -> "opcional si no añade complejidad".	Añadirlo o no.	Decisión de equipo.	Default: NO generarlo (no aporta valor si CI ya lo ejecuta). Si se desea, usar ruff + ruff-format solo.

A8	ADR-007 idempotencia: ¿implementar [domain/share d/value_objec ts/idempotenc y.py] en 0.1 o dejarlo solo en ADR?	Plan actual: solo ADR (0.1 no implementa la entidad).	Decisión de scope 0.1.	Default: solo ADR (mantener 0.1 mínima). Entity + VO = FASE 1.
A9	¿Documentos ADR existen en 0.1 solo como TÍTULOS o con contenido COMPLETO?	El scope dice ADR-001 a ADR-005 (ahora 001-007). ¿Todos los ADRs se escriben completos ahora?	Decisión importante.	Default: Sí completar los 7 ADRs en la 0.1 (secciones Contexto/Alter nativas/Decisi ón/Consecue ncias). Son decisiones que afectan FASE 1.
A10	[application/_ _init__.py] vs. carpetas [commands/q ueries/service s/ports] marcadores vacíos.	Corrección 6 dice no inflar.	Decisión estética.	Default: SOLO [application/_ _init__.py]. Nada más en 0.1.

FIN DOCUMENTO v2.0 - Plan Entrega 0.1 Architectural Baseline (revisado con 18 correcciones arquitectónicas aplicadas).

Espera aprobación explícita antes de proceder a implementar código.